Supporting Information for "Micromechanics of sheared granular layers activated by fluid pressurization"

Hien Nho Gia Nguyen[1], Luc Scholtès[1,2], Yves Guglielmi[3], Frédéric Victor Donzé[4], Zady Ouraga[5], Mountaka Souley[5]

[1]Université de Lorraine|CNRS|GeoRessources, Nancy, France

[2]Université Clermont Auvergne|CNRS|IRD|OPGC|Laboratoire Magmas et Volcans, Clermont-Ferrand, France

[3]Lawrence Berkeley National Laboratory|Energy Geosciences Division, Berkeley, CA, USA

[4]Université Grenoble-Alpes|Université Savoie Mont Blanc|CNRS|IRD|IFSTTAR|ISTerre, Grenoble, France

[5]French National Institute for Industrial Environment and Risks (Ineris), Verneuil-en-Halatte, France

Contents of this file

1. Text S1 to S3
2. Figure S1 to S3
3. Table S1

1. DEM-PFV formulations

The numerical sample consists of smooth spherical particles. The DEM module handles the solid phase while the PFV module handles the fluid phase.[1] The DEM is based on a Lagrangian approach where each particle is identified by its own mass, size and moment of inertia. The computation scheme consists in integrating Newton's second law so as to determine particles motion and interactions one with another. The DEM model is a 3D implementation in the open source code YADE-DEM where interparticle contacts are modeled by linear elastic relationships between forces and interparticle displacements, associated with a slip Coulomb model S1. The contact laws governing the interactions are defined from 3 parameters including the normal stiffness coefficient k_n, the tangent stiffness coefficient k_t and the microscopic friction coefficient μ_p (Cundall & Strack, 1979). The inter-particle contact behavior have two components: (1) the normal contact is governed by an elastic force-displacement relation $\Delta F_n = k_n \Delta\delta_c \geq 0$ where δ_c is the overlapping between two particles in contact; (2) the tangent force is incrementally computed at each numerical time step as $\Delta F_t = k_t \Delta u_t$ and $|F_t| \leq F_n \tan\mu_p$ (Šmilauer et al., 2010). The parameters for the contact laws are given in Table S1. The numerical sample consists of 12,000 spherical particles with a uniform size distribution whose diameters vary between 0.066 mm and 0.133 mm with the density of 2600 kg/m^3. The microscopic friction coefficient μ_p is 0.52. The numerical time step in the simulation is constant, it is calculated as a fraction of the critical time step $\Delta t = 0.5\Delta t_{cr}$ with

$$\Delta t_{cr} = \min_p \left(R_p \sqrt{\rho_p / E_p} \right) \tag{1}$$

where R is the particle radius, ρ is the density and E is the elastic modulus of the particle from which the contact stiffnesses are computed as $K_n = E\pi' R'$, where π' is a constant depending on the algorithm/material, for classical DEM model $\pi' = 2$, R' is half-distance between spheres in contact. In this study, the critical time step is $6.45 \cdot 10^{-5} s$.

For the fluid coupling, the PFV module is able to describe the compressible flow at the particle scale in the viscous regime. A partition of the pore space is obtained by constructing the regular triangulation of the packing. The elementary objects emerging from this procedure are tetrahedrons whose vertices are defined by the centers of gravity of the particles. The space limited by this structure is known as the pore space. For the sake of simplicity, the information in this document is a summary extracted from (Scholtès et al., 2015). More details of the PFV formulation can be found in (Chareyre et al., 2012).

Let us define Ω_i the portion of the pore i occupied by the fluid (Figure S2). Integrating the continuity equation in Ω_i gives:

$$\int_{\Omega_i} \frac{\partial \rho_{\mathrm{f}}}{\partial t} \mathrm{d}V = -\int_{\Omega_i} \nabla \cdot (\rho_{\mathrm{f}} \mathbf{v}) \, \mathrm{d}V \tag{2}$$

where ρ_f is the fluid density and $\mathbf{v}$ is the fluid velocity. Applying the divergence theorem leads Eq. (2) to

$$\int_{\Omega_i} \frac{\partial \rho_{\mathrm{f}}}{\partial t} \mathrm{d}V = -\int_{\partial\Omega_i} \rho_{\mathrm{f}} \mathbf{v} \cdot \mathbf{n} \mathrm{d}S \tag{3}$$

with $\mathbf{n}$ the outward pointing unit vector normal to $\partial\Omega_i$. As the vertices of the elements follow the motion of the particles, the element is deforming at a certain rate. Introducing $\mathbf{u}$ as the velocity of the contour $\partial\Omega_i$, Eq. (3) can be written as:

$$\int_{\Omega_i} \frac{\partial \rho_\mathrm{f}}{\partial t} \mathrm{d}V = -\int_{\partial\Omega_i} \rho_\mathrm{f}(\mathbf{v}-\mathbf{u})\cdot\mathbf{n}\mathrm{d}S - \int_{\partial\Omega_i} \rho_\mathrm{f}\mathbf{u}\cdot\mathbf{n}\mathrm{d}S \tag{4}$$

The bulk modulus K_f relates the time derivatives of ρ_f and P_i as

$$K_\mathrm{f} = \rho_\mathrm{f}\frac{\partial P_i}{\partial \rho_\mathrm{f}}, \tag{5}$$

for a compressible fluid.

For saturated media (degree of saturation $S_r = 1$), K_f is equal to the fluid bulk modulus (e.g. water $K_w = 2.2 \times 10^9$ Pa). If air phase is available in the fluid ($0.9 < S_r < 1$), K_f depends on the gaz content in the fluid mixture:

$$K_\mathrm{f} = \frac{1}{\dfrac{S_r}{K_\mathrm{w}} + \dfrac{1-S_r}{K_\mathrm{a}}} \tag{6}$$

assuming the air distribution within fluid is uniform in all pores, with K_a the bulk modulus of air.

Considering Eq. (5) and neglecting the spatial fluctuation of ρ_f in the vicinity of the domain (small Mach number), Eq. (4) thus becomes

$$\int_{\Omega_i} \frac{1}{K_\mathrm{f}}\frac{\partial P_i}{\partial t}\mathrm{d}V = -\sum_{j=1}^{4}\int_{S_{ij}^f}(\mathbf{v}-\mathbf{u})\cdot\mathbf{n}\mathrm{d}S - \dot{V}_{p,i} \tag{7}$$

where $\dot{V}_{p,i}$ is the time derivative of the pore volume and S_{ij}^f is the intersection of the triangular surface with the fluid domain along each side of the tetrahedron.

The fluxes integrals appearing on the right-hand side of Eq(7) can be linked to the pressure jump between two pores i and j so that the equation becomes

$$\dot{P}_i = -\frac{K_\mathrm{f}}{V_{f,i}} \left[\dot{V}_{p,i} + \sum_{j=1}^{4} k_{ij} \left(P_i - P_j \right) \right] \tag{8}$$

with k_{ij} the conductivity of the thoat between i and j, k_{ij} being computed as a function of the thoat geometry through a generalized Poiseuille's law.

A finite difference discretization of Eq. (8) using a backward Euler scheme for the evaluation of $\dot{P}_i$ gives

$$\sum_{j=1}^{4} k_{ij} P_j^t - \left(\zeta_i + \sum_{j=1}^{4} k_{ij} \right) P_i^t = \dot{V}_{p,i}^{t-1/2} - \zeta_i P_i^{t-1}, \quad \text{with} \quad \zeta_i = \frac{V_{f,i}}{K_\mathrm{f} \Delta t} \tag{9}$$

where the time-centered evaluation of $\dot{V}_{p,i}^{t-1/2}$ is obtained from mid-step velocities of the particles. An implicit scheme is obtained by considering Eq. (9) for all pores at a given time. The associated linear system has to be solved as each time-step of the simulation, linking the pressure field P^t to the deformation rate of the pore space.

In turn, the force exerted by the fluid on each particle p can be deduced using the same expressions as for an incompressible fluid. It is the sum of three terms which are contour integrals of the hydrostatic pressure $\rho_\mathrm{f} g z$ (buoyancy), of the piezometric pressure P and the viscous shear stress $\boldsymbol{\tau}$, respectively:

$$\mathbf{F}^p = \int_{\Gamma_p} \rho_\mathrm{f} g z \mathbf{n} \mathrm{d}S + \int_{\Gamma_p} P \mathbf{n} \mathrm{d}S + \int_{\Gamma_p} \tau \mathbf{n} \mathrm{d}S = \mathbf{F}^{B,p} + \mathbf{F}^{P,p} + \mathbf{F}^{V,p}, \tag{10}$$

Γ_p is the solid surface of the particle p. These forces are determined for time t and integrated into the conventional explicit time-stepping algorithm of the DEM by summing

them together with the contact forces. This sequence of equations (9) and (10) defines a semi-implicit integration scheme.

The fluid pressure is applied from the bottom boundary of the system (the top boundary is set up with a no flow condition) and there is a brief transient response of the sample when the pressure is increased as illustrated by the oscillations observed right after the pressure steps are applied (Figure 2 in the main text). However, due to the relatively high permeability of the medium, the pore pressure is homogeneously distributed after only a few timesteps following the pressure steps.

2. Analysis methods

2.1. Stress evaluation

The boundary stress ($\boldsymbol{\sigma}$) can be calculated from the interactions between the plane and particles in contact. At a selected state, the boundary normal stress is the sum of all plane-particle normal forces divided by the effective area of the plane. Likewise, the boundary shear stress is calculated from the shear components of the plane-particle interactions. The contact stress ($\tilde{\boldsymbol{\sigma}}$) is calculated from all interactions within the sample as $\tilde{\sigma}_{ij} = \frac{1}{V}\sum_b f_i^b l_j^b$ where f is the contact force and l is the branch vector joining the centres of two particles in contact, V is the volume of the sample.

2.2. Extracting sub-sample

While the numerical sample is three dimensional and it has periodic boundaries on the horizontal directions, a vertical slide with thickness of about 15 particles is sufficient for the analysis. This sub-sampling method allows the efficiency of the analysis while maintain the clearance of data visualization by projecting 3D data points on a 2D plan.

2.3. Particle rotation calculation

The accumulated rotation of the particle is calculated from the snapshots of the sample between two stages: the current selected stage versus the initial stage where the fluid injection starts (P = 0 MPa).

2.4. Contact force network

The contact force network is a set of particle-particle interactions visualized based on the position of the grains in contact and the intensity of their local contact force. At a selected stage, normal contact forces of all interaction can be computed as mentioned in Section 1. The line segments represent the vector branch connecting two particles in contact. For the normalization, the max and min of normal contact forces from all three stages are taken into account. Then, the normalization is based on these max and min values, enabling a consistent comparison method.

2.5. Sliding contact distribution

For a particle-particle interaction, when the computed tangent force F_t exceeds the threshold $F_n \tan \mu_p$, the contact is defined as a 'sliding contact'. The position of a sliding contact is the midpoint of two involved particles. It is worth noting that it is expected to have high amount of sliding contacts at the boundaries (top and bottom plates) because of the shear control is governed by the plate movement. Hence, in order to focus better on what happens within the sample, especially around the shear band, the sliding contacts near the boundaries are not displayed. Moreover, in order to highlight the distribution of the sliding contacts within the sample, each map is accompanied by a density histogram (Figure S3).

3. Source codes

The open source software YADE can be found at http://yade-dem.org. The python codes used in this paper include two scripts: script 1 is for the shear test before fluid injection and script 2 is for the fluid injection test. The raw data is available at https://doi.org/10.5061/dryad.wh70rxwnf.

3.1. Shear test

```
from yade import pack,plot,export
import math


sp         = pack.SpherePack()
O.periodic = True


# dimensions of sample (fixed by particle size such as L/D~X - length unit is mm)
DIAMETER  = 1.e-1
RADIUS    = 0.5*DIAMETER
length    = 15*(DIAMETER)*2
height    = length
width     = length
thickness = length/100.


# microproperties
DENS     = 2600
E        = 1e9
P        = 0.25
compFRIC = 1.
FRIC     = 30.
TENS     = 0.
COH      = 0.
```

```
# boundary conditions
PI = 1.e5
conf = 0.
if conf<PI:
  PC = PI
else:
  PC = conf
SN = 5.e6 # normal stress for shearing phase
RATE = 0.02 # shearing rate

# simulation control
DAMPSHEAR = 0.
ITER      = 1e6
VTK       = 10
OUT       = 'l20h20w20_e1e9p025cf1f30_pi1e5pc0sn5e6_v0.02'

#### create sample and loading boxes

O.cell.hSize = Matrix3(length,0,0,0,3*height,0,0,0,width)

O.materials.append(CohFrictMat(isCohesive=True,density=DENS,young=E,poisson=P,
    frictionAngle=radians(0.),normalCohesion=1e100,shearCohesion=1e100,label='boxMat'))
O.materials.append(CohFrictMat(isCohesive=True,density=DENS,young=E,poisson=P,
    frictionAngle=radians(compFRIC),normalCohesion=TENS,shearCohesion=COH,label='
    sphereMat'))

upBox  = utils.box(center=(length/2.0,2*height+thickness/2.0,width/2.0),orientation=
    Quaternion(1,0,0,0),extents=(length,thickness/2.,width),fixed=1,wire=False,color
    =(1,0,0),material='boxMat')
```

```
lowBox = utils.box(center=(length/2.0,height-thickness/2.0,width/2.0),orientation=
    Quaternion(1,0,0,0),extents=(length,thickness/2.,width),fixed=1,wire=False,color
    =(1,0,0),material='boxMat')
O.bodies.append([upBox,lowBox])

sp.makeCloud((0,height+1.5*RADIUS,0),(length,2*height-1.5*RADIUS,width),rMean=RADIUS,
    rRelFuzz=0.2,periodic=True)
O.bodies.append([utils.sphere(s[0],s[1],color=(0,0,1),material='sphereMat') for s in sp])

effCellVol = (O.bodies[0].state.pos[1]-O.bodies[1].state.pos[1])*O.cell.hSize[0,0]*O.cell
    .hSize[2,2]
volRatio   = (O.cell.hSize[0,0]*O.cell.hSize[1,1]*O.cell.hSize[2,2])/effCellVol

#### engines

flow = PeriodicFlowEngine(
    isActivated         = 0,
    useSolver           = 3,
    defTolerance        = -1,
    meshUpdateInterval  = 1000,
    duplicateThreshold  = 0.5,
    boundaryUseMaxMin   = [0,0,1,1,0,0],
    wallIds             = [-1,-1,1,0,-1,-1],
    wallThickness       = thickness,
    bndCondIsPressure   = [0,0,0,0,0,0],
    bndCondValue        = [0,0,0,0,0,0],
    permeabilityFactor  = 1,
    viscosity           = 1,
    fluidBulkModulus    = 2.2e9,
    label               = 'flowEng'
  )
```

```
O.engines = [
  ForceResetter()
  ,InsertionSortCollider([Bo1_Box_Aabb(),Bo1_Sphere_Aabb()],verletDist=-0.1,
      allowBiggerThanPeriod=True)
  ,InteractionLoop(
    [Ig2_Sphere_Sphere_ScGeom6D(),Ig2_Box_Sphere_ScGeom6D()],
    [Ip2_CohFrictMat_CohFrictMat_CohFrictPhys()],
    [Law2_ScGeom6D_CohFrictPhys_CohesionMoment(traceEnergy=True,label='contactLaw')]
  )
  ,flow
  ,PeriTriaxController(
    dynCell       = True,
    mass          = 10,
    maxUnbalanced = 1e-3,
    relStressTol  = 1e-4,
    stressMask    = 7,
    goal          = (-PI/volRatio,-PI/volRatio,-PI/volRatio),
    globUpdate    = 1,
    maxStrainRate = (1,1,1),
    doneHook      = 'triaxDone()',
    label         = 'triax'
    )
  ,GlobalStiffnessTimeStepper(active=1,timeStepUpdateInterval=100,
      timestepSafetyCoefficient=0.8,defaultDt=utils.PWaveTimeStep(),label='timeStepper')
  ,NewtonIntegrator(damping=0.3,label='newton')
  ,PyRunner(command='dataRecorder()',iterPeriod=10,label='recData',dead=True)
  ,PyRunner(iterPeriod=1,command='saveFlowVTK()',label='saveFluid',dead=True)
  ,VTKRecorder(fileName=OUT+'.',iterPeriod=1,skipNondynamic=1,recorders=['spheres','
      colors','velocity','bstresses'],label='saveSolid',dead=True)
]
```

```
def saveFlowVTK():
  flow.saveVtk(folder='vtkFiles_'+OUT)


def dataRecorder():
  h              = vol=vol_s=nb_s=0.
  h              = O.bodies[0].state.pos[1]-O.bodies[1].state.pos[1]
  vol            = h*O.cell.hSize[0,0]*O.cell.hSize[2,2]
  contactStress = getStress(vol)
  for o in O.bodies:
      if isinstance(o.shape,Sphere) and o.shape.color[0]!=1:
        nb_s += 1
        vol_s += 4.*pi/3.*(o.shape.radius)**3
  n            = 1-vol_s/vol
  nbFrictCont = 0.
  for i in O.interactions:
    if i.isReal and i.phys.cohesionBroken:
      nbFrictCont+=1
  plot.addData(
    iter              = O.iter,
    stress_upWall0   = abs(O.forces.f(0)[0]/(O.cell.hSize[0,0]*O.cell.hSize[2,2])),
    stress_upWall1   = abs(O.forces.f(0)[1]/(O.cell.hSize[0,0]*O.cell.hSize[2,2])),
    stress_upWall2   = abs(O.forces.f(0)[2]/(O.cell.hSize[0,0]*O.cell.hSize[2,2])),
    contactStress00 = (contactStress[0,0]),
    contactStress01 = (contactStress[0,1]),
    contactStress02 = (contactStress[0,2]),
    contactStress10 = (contactStress[1,0]),
    contactStress11 = (contactStress[1,1]),
    contactStress12 = (contactStress[1,2]),
    contactStress20 = (contactStress[2,0]),
    contactStress21 = (contactStress[2,1]),
```

```
    contactStress22 = (contactStress[2,2]),
    xW              = O.bodies[0].state.pos[0],
    height          = h,
    volume          = vol,
    porosity        = n,
    k               = 2.0*nbFrictCont/nb_s,
    Ek              = kineticEnergy(),
    unbF            = unbalancedForce()
    )


phase=0
def triaxDone():
  global phase
  volRatio=(O.cell.hSize[0,0]*O.cell.hSize[1,1]*O.cell.hSize[2,2])/((O.bodies[0].state.
      pos[1]-O.bodies[1].state.pos[1])*O.cell.hSize[0,0]*O.cell.hSize[2,2])
  if phase==0:
    h=O.bodies[0].state.pos[1]-O.bodies[1].state.pos[1]
    vol=h*O.cell.hSize[0,0]*O.cell.hSize[2,2]
    contactStress=getStress(vol)
    vol_s=Rmean=Rmax=nbSph=0
    Rmin=1e6
    for o in O.bodies:
      if isinstance(o.shape,Sphere):
        nbSph+=1
        Rmean+=o.shape.radius
        if o.shape.radius>Rmax: Rmax=o.shape.radius
        if o.shape.radius<Rmin: Rmin=o.shape.radius
        vol_s += 4.*pi/3.*(o.shape.radius)**3
    Rmean=Rmean/nbSph
    n = 1-vol_s/vol
```

```
    print('DONE! iter =',O.iter,'| sample generated: nb spheres =',nbSph,', Rmean =',
        Rmean,', Rratio =',Rmax/Rmin,', porosity =',n)
    print('Changing contact properties now')
    utils.setContactFriction(radians(FRIC))
    print('APPLYING CONFINING PRESSURE: sx,sy and sz will go to PC =',PC)
    triax.goal=(-PC/volRatio,-PC/volRatio,-PC/volRatio)
    phase+=1
  elif phase==1:
    print('DONE! iter =',O.iter,'| isotropic confinement done: stresses =',volRatio*triax
        .stress)
    triax.dead=True
    O.pause()

#### Initialization
print('SAMPLE PREPARATION!')

#recData.dead=False # uncomment if you want to record what is happening during
    preparation of sample (isotropic compaction)
O.run(1000000,1)
saveSolid.dead=False
O.step()
saveSolid.dead=True
O.save(OUT+'_isoConfined_'+str(O.iter)+'.yade.gz')

print('Normal stress (platen) =',O.forces.f(0)[1]/(O.cell.hSize[0,0]*O.cell.hSize[2,2]))
print('Normal stress (contacts) =',getStress((O.bodies[0].state.pos[1]-O.bodies[1].state.
    pos[1])*O.cell.hSize[0,0]*O.cell.hSize[2,2])[1,1])

#### Applying normal stress
print('NORMAL LOADING! iter =',O.iter)
```

```
stage = 0
stiff = fnPlaten=currentSN=0.
def servo():
  global stage,stiff,fnPlaten,currentSN
  if stage==0:
    currentSN=O.forces.f(0)[1]/(O.cell.hSize[0,0]*O.cell.hSize[2,2])
    unbF=unbalancedForce()
    boundaryVel=copysign(min(0.1,abs(0.5*(currentSN-SN))),currentSN-SN)
    O.bodies[0].state.vel[1]=boundaryVel
    if ( (abs(currentSN-SN)/SN)<0.001 and unbF<0.001 ):
      stage+=1
      fnPlaten=O.forces.f(0)[1]
      print('Normal stress =',currentSN,' | unbF =',unbF)
      for i in O.interactions.withBody(O.bodies[0].id):
        stiff+=i.phys.kn
      print('Normal stiffness =',stiff)
      print('DONE! iter =',O.iter)
      O.pause()
  if stage==1:
    fnDesired = SN*(O.cell.hSize[0,0]*O.cell.hSize[2,2])
    boundaryVel = copysign(min(0.1,abs(0.35*(O.forces.f(0)[1]-fnDesired)/stiff/O.dt)),O.
        forces.f(0)[1]-fnDesired)
    O.bodies[0].state.vel[1]=boundaryVel

O.engines = O.engines[:4]+[PyRunner(command='servo()',iterPeriod=1,label='servo')]+O.
    engines[4:]

O.trackEnergy = True

O.run(1000000,1)
print('STABILIZING! iter=',O.iter)
```

```
O.run(1000,1)
dxi  = 4*(2*RADIUS)
n    = int(length/dxi)
xmin = 1e6
for i in range(0,n):
  for o in O.bodies:
    if o.id>1:
      if o.state.pos[0]<xmin: xmin=o.state.pos[0]
      if (o.state.pos[0]>=i*dxi) and (o.state.pos[0]<((i+0.5)*dxi)):
        o.shape.color[1]=1
markerSpheres=[]
for o in O.bodies:
  if (o.state.pos[0]>(xmin+0.5*O.cell.hSize[0,0]-1*RADIUS)) and (o.state.pos[0]<(xmin
      +0.5*O.cell.hSize[0,0]+2*RADIUS)):
    o.shape.color[2]=0
    markerSpheres.append(o)

def markerRecorder():
  global intOnFracPlane
  inFile=open(OUT+'_markerPosAndVel_'+str(O.iter),'a')
  for s in markerSpheres:
    inFile.write( str(s.state.pos[0]) + '\t' + str(s.state.pos[1]) + '\t' + str(s.state.
        pos[2]) + '\t' + str(s.state.vel[0]) + '\t' + str(s.state.vel[1]) + '\t' + str(s.
        state.vel[2]) + '\n')
  inFile.close()

O.engines = O.engines+[PyRunner(command='markerRecorder()',iterPeriod=1,label='recMark',
    dead=True)]

print('Normal stress (platen) = ',O.forces.f(0)[1]/(O.cell.hSize[0,0]*O.cell.hSize[2,2]))
```

```
print('Normal stress (contacts) = ',getStress((O.bodies[0].state.pos[1]-O.bodies[1].state
    .pos[1])*O.cell.hSize[0,0]*O.cell.hSize[2,2])[1,1])

#### preparing for shearing
Gl1_Sphere.stripes=1

## gluing particles in contact with the walls
for i in O.interactions:
    if i.isReal:
      if isinstance(O.bodies[i.id1].shape,Box):
        O.bodies[i.id2].shape.color[0]=1
      if isinstance(O.bodies[i.id2].shape,Box):
        O.bodies[i.id1].shape.color[0]=1

for i in O.interactions:
    if i.isReal and ( O.bodies[i.id1].shape.color[0]==1 and O.bodies[i.id2].shape.color
        [0]==1 ):
      O.bodies[i.id1].mat.normalCohesion=O.bodies[i.id1].mat.normalCohesion
      O.bodies[i.id2].mat.normalCohesion=O.bodies[i.id1].mat.normalCohesion
      O.bodies[i.id1].mat.shearCohesion=O.bodies[i.id1].mat.shearCohesion
      O.bodies[i.id2].mat.shearCohesion=O.bodies[i.id1].mat.shearCohesion
      i.phys.initCohesion=True

saveSolid.dead=False
recMark.dead=False
O.step()
saveSolid.dead=True
recMark.dead=True
O.save(OUT+'_normallyLoaded_'+str(O.iter)+'.yade.gz')
if recData.dead==False: plot.saveDataTxt(OUT)
```

```
#### shearing


recData.dead=False
recData.iterPeriod=int(ITER/10000.)
recMark.dead=False
recMark.iterPeriod=int(100000)
newton.damping=DAMPSHEAR
saveSolid.dead=False
saveSolid.iterPeriod=int(ITER/VTK)
shearVel=0
iterShear=recIter=O.iter
while 1:
  if ( O.iter >= int(recIter+100000) ):
    recIter=O.iter
    plot.saveDataTxt(OUT)
    O.save(OUT+'_sheared_'+str(O.iter)+'.yade.gz')
  if ( O.iter >= int(iterShear+ITER) ):
    print('iter =',O.iter,' -> END!')
    plot.saveDataTxt(OUT)
    O.save(OUT+'_sheared_'+str(O.iter)+'.yade.gz')
    O.save('laststate.yade.gz')
    sys.exit(0)
  O.run(int(10),1)
  if shearVel<RATE:
    shearVel+=(RATE/100.)
  O.bodies[0].state.vel[0]=shearVel
```

3.2. Fluid injection test

```
from yade import pack,plot,export


prevSim= 'laststate'
```

```
O.load(prevSim+'.yade.gz')


sigmaN = 5.e6

rTau   = 0.8

output = prevSim+'_output'


## time stepper (needed if time control required)

timer        = utils.typedEngine('GlobalStiffnessTimeStepper')

timer.active = False

timer.dead   = True

O.dt         = 0.5*utils.PWaveTimeStep()


## vtk recorder

saveSolid.dead           = True

saveSolid.iterPeriod     = int(1)

saveSolid.fileName       = output+'.'

saveSolid.skipNondynamic = 1

saveSolid.recorders      = ['spheres','colors','velocity','bstresses']


def saveFlowVTK():

 flowEng.saveVtk(folder=output+'_vtkFluid')

saveFluid.dead       = True

saveFluid.iterPeriod = int(1)


## data recorder

inputP=0.

def dataRecorder():

  global inputP

  h=vol=vol_s=nb_s=0.

  h=O.bodies[0].state.pos[1]-O.bodies[1].state.pos[1]

  vol=h*O.cell.hSize[0,0]*O.cell.hSize[2,2]
```

```
contactStress=getStress(vol)
for o in O.bodies:
    if isinstance(o.shape,Sphere) and o.shape.color[0]!=1:
      nb_s += 1
      vol_s += 4.*pi/3.*(o.shape.radius)**3
n = 1-vol_s/vol
nbFrictCont=0.
for i in O.interactions:
  if i.isReal and i.phys.cohesionBroken:
    nbFrictCont+=1
plot.addData(
  iter              = O.iter
  ,stress_upWall0   = abs(O.forces.f(0)[0]/(O.cell.hSize[0,0]*O.cell.hSize[2,2]))
  ,stress_upWall1   = abs(O.forces.f(0)[1]/(O.cell.hSize[0,0]*O.cell.hSize[2,2]))
  ,stress_upWall2   = abs(O.forces.f(0)[2]/(O.cell.hSize[0,0]*O.cell.hSize[2,2]))
  ,contactStress00 = (contactStress[0,0])
  ,contactStress01 = (contactStress[0,1])
  ,contactStress02 = (contactStress[0,2])
  ,contactStress10 = (contactStress[1,0])
  ,contactStress11 = (contactStress[1,1])
  ,contactStress12 = (contactStress[1,2])
  ,contactStress20 = (contactStress[2,0])
  ,contactStress21 = (contactStress[2,1])
  ,contactStress22 = (contactStress[2,2])
  ,xW               = O.bodies[0].state.pos[0]
  ,height           = h
  ,volume           = vol
  ,porosity         = n
  ,k                = 2.0*nbFrictCont/nb_s
  ,Ek               = kineticEnergy()
  ,unbF             = unbalancedForce()
```

```
  )

## positions recorder
markerSpheres=[]
for o in O.bodies:
  if o.shape.color[2]==0:
    markerSpheres.append(o)

print('nb or markers=',len(markerSpheres))

def markerRecorder():
  global intOnFracPlane
  inFile=open(output+'_markerPosAndVel_'+str(O.iter),'a')
  for s in markerSpheres:
    inFile.write( str(s.state.pos[0]) + '\t' + str(s.state.pos[1]) + '\t' + str(s.state.
        pos[2]) + '\t' + str(s.state.vel[0]) + '\t' + str(s.state.vel[1]) + '\t' + str(s.
        state.vel[2]) + '\n')
  inFile.close()

recMark.dead=True
recMark.iterPeriod=1

#### servocontrol of top plate:

# we need one step to have a force on platen
for o in O.bodies:
  o.dynamic=False
saveSolid.dead=False
recMark.dead=False
O.run(1,1)
saveSolid.dead=True
```

```
recMark.dead=True
O.save(output+'_'+str(O.iter)+'.yade.gz')
for o in O.bodies:
  if isinstance(o.shape,Sphere): o.dynamic=True

nStiff=0
for i in O.interactions.withBody(O.bodies[0].id):
  nStiff+=i.phys.kn
print('normal stiffness=',nStiff)
initFnPlaten=O.forces.f(0)[1]
initSigmaN=initFnPlaten/(O.cell.hSize[0,0]*O.cell.hSize[2,2])
print('normal stress (platen) =',initSigmaN)

def servo():
  fnDesired=sigmaN*(O.cell.hSize[0,0]*O.cell.hSize[2,2])
  nBoundaryVel=copysign(min(0.1,abs(0.35*(O.forces.f(0)[1]-fnDesired)/nStiff/O.dt)),O.
      forces.f(0)[1]-fnDesired)
  #nBoundaryVel=copysign(abs(0.35*(O.forces.f(0)[1]-fnDesired)/nStiff/O.dt),O.forces.f(0)
      [1]-fnDesired)
  O.bodies[0].state.vel[1]=nBoundaryVel

#### Stabilization
print('stabilizing | iter=',O.iter)
newton.damping=0.3
O.bodies[0].state.vel[0]=0
recData.dead=False
recData.iterPeriod=1

O.run(int(1e3),1)
saveSolid.dead=False
recMark.dead=False
```

```
O.run(1,1)
saveSolid.dead=True
recMark.dead=True
plot.saveDataTxt(output)
O.save(output+'_'+str(O.iter)+'.yade.gz')

#### shear stress control

sStiff=0
for i in O.interactions.withBody(O.bodies[0].id):
  sStiff+=i.phys.ks
print('shear stiffness=',sStiff)
initFsPlaten=O.forces.f(0)[0]
initTau=initFsPlaten/(O.cell.hSize[0,0]*O.cell.hSize[2,2])
print('shear stress (platen) =',initTau)

maxSBVel=0.1
def servoShear():
  global maxSBVel
  fsDesired=rTau*initTau*(O.cell.hSize[0,0]*O.cell.hSize[2,2])
  sBoundaryVel=copysign(min(maxSBVel,abs(0.35*(O.forces.f(0)[0]-fsDesired)/sStiff/O.dt)),
      O.forces.f(0)[0]-fsDesired)
  O.bodies[0].state.vel[0]=sBoundaryVel

O.engines = O.engines[:5]+[PyRunner(command='servoShear()',iterPeriod=1,label='servoShear
    ')]+O.engines[5:]

print('shear control now | iter=',O.iter)

while 1:
  O.run(100,1)
```

```
  if (unbalancedForce()<0.001) and ( ((abs(O.forces.f(0)[0]/(O.cell.hSize[0,0]*O.cell.
      hSize[2,2])) - abs(rTau*initTau))/abs(rTau*initTau))<0.001 ):
    print ('stress state reached| iter=',O.iter,' | shear stress (platen) =',O.forces.f
        (0)[0]/(O.cell.hSize[0,0]*O.cell.hSize[2,2]))
    print('stabilizing')
    O.run(int(1e3),1)
    plot.saveDataTxt(output)
    saveSolid.dead=False
    recMark.dead=False
    O.run(1,1)
    saveSolid.dead=True
    recMark.dead=True
    O.save(output+'_'+str(O.iter)+'.yade.gz')
    break

#### injection
print('FLUID NOW | iter=',O.iter)
maxSBVel                        = 1e3
flowEng.isActivated             = 1
flowEng.bndCondIsPressure       = [0,0,1,0,0,0]
deltaP                          = 1e5
inputP                          = deltaP
flowEng.bndCondValue            = [0.,0.,inputP,0.,0.,0.]
flowEng.fluidBulkModulus        = 2.2e9
flowEng.permeabilityFactor      = 1
flowEng.viscosity               = 1
newton.damping                  = 0.

iterInit       = injectIter=recIter=O.iter
deltaIter      = 3000
iterMax        = 20*deltaIter
```

```
runningIter   = O.iter


while 1 :
  O.run(1,1)
  if ( O.iter >= int(injectIter+deltaIter) ):
    injectIter = O.iter
    inputP     +=deltaP
    flowEng.bndCondValue=[0.,0.,inputP,0.,0.,0.]
    flowEng.updateBCs()
    print('updateBCs! inputP=',inputP)
  if ( O.iter >= int(recIter+int(deltaIter/2.)) ):
    recIter        = O.iter
    saveSolid.dead = False
    saveFluid.dead = False
    recMark.dead   = False
    O.run(1,1)
    saveSolid.dead = True
    saveFluid.dead = True
    recMark.dead   = True
    plot.saveDataTxt(output)
    O.save(output+'_'+str(O.iter)+'.yade.gz')
    print('saving data!')
  if ( O.iter >= int(iterInit+iterMax) ):
    print('iter=',O.iter,' -> END!')
    plot.saveDataTxt(output)
    O.save(output+'_'+str(O.iter)+'.yade.gz')
    break
```

References

Chareyre, B., Cortis, A., Catalano, E., & Barthélemy, E. (2012). Pore-Scale Modeling

of Viscous Flow and Induced Forces in Dense Sphere Packings. *Transport in Porous Media*, *92*(2), 473-493. doi: 10.1007/s11242-011-9915-6

Cundall, P. A., & Strack, O. D. L. (1979). A discrete numerical model for granular assemblies. *Géotechnique*, *29*(1), 47-65. doi: 10.1680/geot.1979.29.1.47

Scholtès, L., Chareyre, B., Michallet, H., Catalano, E., & Marzougui, D. (2015). Modeling wave-induced pore pressure and effective stress in a granular seabed. *Continuum Mechanics and Thermodynamics*, *27*(1), 305-323. doi: 10.1007/s00161-014-0377-2

Šmilauer, V., Catalano, E., Chareyre, B., Dorofeenko, S., Duriez, J., Gladky, A., ... others (2010). Yade reference documentation. *Yade Documentation*, *474*(1).

Notes

1. DEM: discrete elements method, PFV: pore-scale finite volume

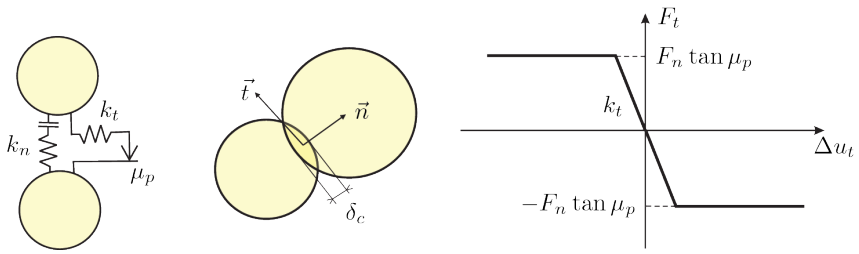

Figure S1. Contact laws governing the interactions.

Table S1. Model properties

Particles and contacts	**value**	**Bulk (steady-state)**	**value**
Particle diameter D_p	$0.1 \pm 0.033\,\mathrm{mm}$	Porosity n	0.438
Particle density ρ_p	$2{,}600\,\mathrm{kg/m^3}$	Permeability $k_x = k_z = 2k_y$	$2.4 \cdot 10^{-11}\,\mathrm{m^2}$
Normal stiffness $\frac{k_n}{D_p}$	$1\,\mathrm{GPa}$	Normal stiffness K_n	$20.9\,\mathrm{MPa} \cdot \mathrm{m}$
Shear stiffness $\frac{k_s}{D_p}$	$0.25\,\mathrm{GPa}$	Shear stiffness K_s	$5\,\mathrm{MPa} \cdot \mathrm{m}$
Friction coefficient μ_p	0.52	Friction coefficient μ	0.38 ± 0.01

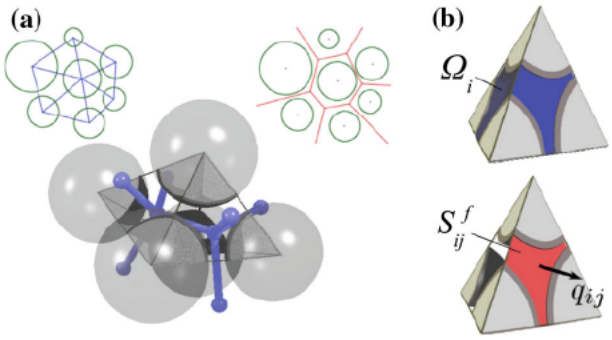

Figure S2. (a) Regular triangulation and its dual Voronoi's graph in two dimensional and in three dimensional. (b) Elementary fluid domain (tetrahedron) in triangulated sphere assembly (Scholtès et al., 2015).

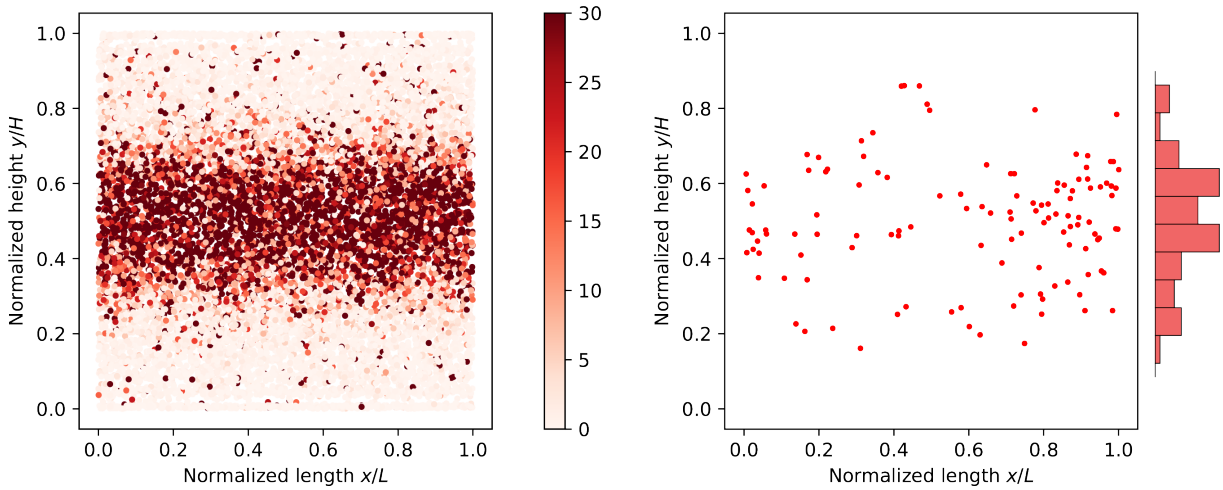

Figure S3. Rotation map and sliding contact map at the steady state chosen from the shear test.